Model Performance Evaluation Framework

Technical Report

Date: Sep 27, 2026

Team: Saniya Shaikh · Vinay Bommenahalli Gurubasappa

Sakshi Manjrekar · Shruti Bhandari

(M.Sc. Applied Data Science & AI)

University: SRH Heidelberg

Supervisor: Prof. Dr.-Ing. Swati Chandna

Prepared in Collaboration with Deutsche Bank

Acknowledgments

The authors thank Ashwani-Kumar Sadana and Michael-C Meier of Deutsche Bank for their guidance, feedback, and support throughout this project, and Prof. Dr.-Ing. Swati Chandna for her supervision of the work.

Table of Contents

Abstract.....4
1. Introduction.....4
2. Theoretical Foundations.....5
3. System Architecture.....9
4. Data Harmonization Module.....11
5. Similarity Search Engine.....14
6. LLM Integration Layer.....16
7. User Interface Design.....17
8. Configuration and Reliability Engineering.....19
9. Results and Demonstration.....20
10. Limitations and Future Work.....24
11. Conclusion.....25
12. Implementation and Deployment Guide.....26
Appendix A: Project Structure Reference.....29

Abstract

This report documents the design, theory, and implementation of the Similarity Scoring Engine, the core component of the Model Performance Evaluation Framework built for this Deutsche Bank–linked academic collaboration project. The system addresses a specific evaluation problem: when testing a financial-analytics assistant, an evaluator needs to know whether the question they are about to ask has effectively been asked before, in different words, so that prior answers can be reused for consistency checking and so evaluation coverage across a historical question log (approximately 334 unique questions across 116 conversations, as of the final submitted dataset; see §4.5) can be measured systematically. The engine takes one new question typed by an evaluator, compares it against the entire harmonized historical dataset, and returns every question judged a genuine semantic match, each carrying a continuous 0.0–1.0 similarity score and a short, human-readable justification.

Architecturally, the system is a two-stage pipeline. The first stage is a deterministic **text harmonization layer**: a dictionary-driven canonicalization pass that rewrites messy, real-world financial phrasing ("C/I ratio", "trade finance", "mid 2025") into a single fixed vocabulary ("Cost-Income Ratio", "Transaction Banking", "Q2 2025"), applied identically to both the historical dataset and any new incoming question so that downstream comparison is apples-to-apples. As of submission, the harmonization dictionary comprises 423 entity entries and 204 time format entries, compiled into 3,869 unique regex patterns covering financial metrics, division names, regulators, time periods, and region names. The second stage is an **LLM-as-judge retrieval layer**: rather than computing embeddings and cosine similarity, or matching keywords, the entire harmonized dataset is handed to a large language model (Google's Gemini 3.8 Flash, accessed through OpenRouter's OpenAI-compatible API) in a single call, guided by a natural-language instruction set that asks the model to reason as a senior financial analyst rather than apply a mechanical scoring rubric. Results are surfaced through a Dash + Mantine web interface designed to be legible to non-technical reviewers: plain-language match strength, "why it matches" reasoning chips, and friendly, audience-appropriate error handling.

This document is written to stand on its own. It explains not only *what* was built but the underlying theory (text canonicalization, the tradeoffs of an LLM-as-judge design versus embedding-based retrieval, structured-output generation, and rate-limiting/reliability engineering) needed to understand *why* the system is built the way it is.

1. Introduction

1.1 Problem Statement

Building and maintaining an evaluation framework for a financial-analytics assistant requires a historical log of test questions and their answers. Over the life of such a project, that log grows to hundreds of entries, contributed by multiple evaluators, phrased in inconsistent language. A recurring, costly problem emerges: an evaluator sits down to write a new test question with no reliable way to know whether an equivalent question already exists in the log, just worded differently: "C/I ratio for CB" versus "Cost-to-Income ratio for Corporate Banking" versus "cost efficiency for corp bank" are, semantically, the same test case. Without a way to detect this, three problems compound over time: evaluation coverage becomes impossible to measure honestly (the log looks larger and more diverse than it is), duplicate effort is spent writing and grading questions that already have an answer on file, and inconsistency creeps in when the same underlying question receives different answers on different occasions because no one realized it had been asked before.

A naïve solution: exact string matching, or simple keyword overlap, fails immediately against this kind of data. Real evaluator phrasing varies in abbreviation ("ROE" vs "equity return" vs "return for shareholders"), in time-period notation ("Q3 2025" vs "3Q25" vs "Jul-Sep 2025"), in division naming ("CB" vs "corp bank" vs "wholesale banking"), and in the underlying request structure itself. What is needed is a system that understands *semantic* equivalence, the same division(s), the same metric(s), the same time period, and the same underlying ask, despite arbitrarily different surface wording.

1.2 Project Context

This system was built as part of an academic collaboration project modeling the kind of internal evaluation tooling a financial institution (in this case, framed around Deutsche Bank's organizational structure, divisions such as Corporate Banking, Investment Banking, Private Banking, Asset Management, and metrics such as Return on Equity, Cost-Income Ratio, and Net Interest Margin) would need to responsibly evaluate an LLM-based analytics assistant before relying on it. The **Similar Question Search** tool documented here is the component of that broader Model Performance Evaluation Framework (MEF) responsible specifically for duplicate/similar-question detection. It is designed to be used interactively by an evaluator during log curation, and its output (match scores and reasoning) is itself a small demonstration of the same LLM-as-judge pattern the wider evaluation framework studies.

1.3 Scope and Goals of This Report

This report documents the final, submitted state of the project. It is written at a level of technical detail sufficient for a reader with no prior exposure to the codebase to understand both *what* the system does and *why* each design decision was made, including the underlying theory, not just an inventory of files. It covers, in order: the theoretical foundations the design rests on (Section 2); the end-to-end system architecture (Section 3); each of the four major code modules in turn, data harmonization (Section 4), the similarity search engine (Section 5), the LLM integration layer (Section 6), and the user interface (Section 7); configuration and reliability engineering (Section 8); a walkthrough of the system in use (Section 9); known limitations and future work (Section 10); a conclusion (Section 11); a standalone, step-by-step implementation and deployment guide (Section 12); and an appendix mapping the full project file structure (Appendix A).

1.4 How to Read This Report

Readers already familiar with the codebase may wish to skip directly to Sections 4–6 for implementation detail. Readers who want to understand the reasoning behind the architecture before seeing the code should read Section 2 (Theoretical Foundations) and Section 3 (System Architecture) first, as later sections assume familiarity with the concepts introduced there. Readers who want to run the system themselves rather than study its design can skip directly to Section 12 (Implementation and Deployment Guide), which is self-contained and does not require having read the rest of the report first.

2. Theoretical Foundations

2.1 Text Canonicalization and Normalization Theory

Canonicalization is the process of mapping every member of a set of equivalent representations to one single, agreed-upon form. It is a foundational technique in information retrieval: search engines lowercase and stem words for the same reason a phone system normalizes "+1 (555) 012-3456" and "555.012.3456" to one internal format before comparing them; comparison is only meaningful once both sides speak the same vocabulary.

Formally, a canonicalization function C maps a domain of surface strings S to a smaller codomain of canonical strings K, such that for any two strings $s_1, s_2 \in S$ that a domain expert would judge equivalent, $C(s_1) = C(s_2)$. In this project, S is the space of ways a Deutsche Bank evaluator might phrase a division, metric, or time period ("CB", "corp bank", "corporate division", "wholesale banking" → all map to the single canonical token "Corporate Banking"), and K is the small, fixed vocabulary defined in the harmonization dictionaries (Section 4). The critical design property is that C must be applied identically and deterministically to every piece of text that will later be compared, both the historical dataset and any new incoming question, otherwise the very mismatch canonicalization is meant to eliminate is silently reintroduced on only one side of the comparison.

Canonicalization here is implemented as **dictionary-driven alias substitution**: a lookup table of (alias → canonical) pairs, compiled into pattern-matching rules, applied via find-and-replace across free text. This is distinct from *stemming* (reducing "running"/"runs"/"ran" to a common root by grammatical rule) or *lemmatization* (reducing to a dictionary headword), those are general-purpose, rule-based techniques for inflection. Alias substitution, by contrast, is a domain-specific, enumerated mapping: it requires no linguistic rule because every valid alias for every canonical term is listed explicitly in advance. This tradeoff is deliberate, it sacrifices generality (an alias not anticipated in the dictionary will not be resolved) for precision and predictability (a listed alias is *always* resolved correctly, with no risk of an automatic rule over- or under-generalizing on financial jargon that a general-purpose NLP stemmer was never trained on).

2.2 Lexical Similarity vs. Semantic Similarity

Text-similarity techniques fall broadly into two families.

Lexical similarity measures overlap in surface form: shared words, shared character n-grams, edit distance. Techniques such as TF-IDF cosine similarity, Jaccard similarity over token sets, or Levenshtein distance all operate purely on the strings themselves, with no model of meaning. Lexical methods are fast and require no external model, but they systematically fail on paraphrase: "How does CB's ROE compare to Markets?" and "Corporate Banking versus Investment Banking on equity return" share almost no surface tokens despite asking, semantically, the same question, while two lexically similar strings can ask about entirely different divisions or periods and still score a high lexical overlap.

Semantic similarity measures whether two pieces of text mean the same thing, independent of exact wording. This is where dense vector embeddings (e.g. sentence-transformer models) are conventionally used: each text is mapped to a point in a high-dimensional vector space such that semantically similar texts land near each other, and similarity is computed as cosine similarity between vectors. This project's design choice, explained in §2.3 below, is to skip the embedding step entirely and use an LLM directly as the semantic judge. Canonicalization (§2.1) is what makes this tractable: by resolving surface-level lexical variance *before* the semantic comparison, the remaining comparison problem is closer to genuine semantic judgment (does the same division/metric/period/ask appear) rather than being confounded by unresolved abbreviations.

2.3 The LLM-as-Judge Retrieval Paradigm

"LLM-as-judge" is an emerging retrieval and evaluation pattern in which a large language model is given a query and a set of candidates directly in its prompt context, and asked to reason about and score each candidate's relevance in natural language, rather than the conventional pipeline of embedding both sides into vectors and ranking by a distance metric.

Why an LLM instead of embeddings + cosine similarity here? The matching criterion in this project is not generic semantic closeness; it is a compound, rule-governed judgment: do the divisions match (exactly, partially, or not at all), does the metric match (exactly or via a closely related metric), does the time period match (exact or approximately overlapping), and does the underlying ask match (same type of request: ranking, comparison, flag). This is closer to *structured, multi-factor relevance judgment* than to undifferentiated "topic similarity," and it is exactly the kind of task an embedding model's single similarity score cannot express, a cosine similarity of 0.87 does not tell you *which* factor is off, and two questions that overlap heavily in vocabulary but differ in one critical factor (e.g. same divisions and metric, but a different quarter) are not distinguishable from two genuinely equivalent questions using vector distance alone. An LLM given explicit natural-language criteria (see §5.2 and the instructions file discussed in Section 5) can reason about each factor individually and produce both a score *and* a stated justification a human reviewer can audit.

Tradeoffs accepted by this design:

- *Cost and latency*: an LLM call is slower and more expensive per query than a vector similarity lookup, which is why the dataset is capped at a size (~334 questions, ~10,000–12,000 tokens of context) that keeps a single call's cost and latency acceptable (see README's own cost note: one search ≈ one Gemini call, ~10,000–12,000 tokens).
- *Non-determinism*: unlike a fixed distance metric, an LLM's judgment can vary slightly between calls on borderline cases. This project accepts this in exchange for reasoning quality, and the continuous 0–1 scoring rubric (§5.2) is deliberately designed to reduce (though not eliminate) this by anchoring the model to explicit, factor-by-factor scoring criteria rather than leaving "similar" undefined.
- *No pre-filtering*: because the LLM sees the *entire* dataset per query rather than a shortlist retrieved by a cheaper first-pass method (a common two-stage "retrieve-then-rerank" pattern in production semantic search systems), the design depends on the LLM being reliable at exhaustively scanning a bounded, moderately-sized candidate set, which the instructions file explicitly compensates for (§5.2, "do not stop at the first match").

2.4 Structured Output Generation

A language model's native output is unstructured free text. For this system's output (a list of match records with typed fields, conversation ID, question number, score, reasoning) to be consumed programmatically by the UI, the model's output must be constrained to a predictable, parseable shape.

The technique used here is **schema-constrained generation**, exposed by OpenAI-compatible chat completion APIs as the `response_format` parameter with `type: "json_schema"`. A JSON Schema is supplied describing the exact object shape expected (see Section 6.2): a top-level object with a `matches` array, each element itself an object with five required, typed fields (`conversation_id`, `question_number`, `matched_question_text`, `similarity_score`, `reasoning`), and `additionalProperties: false` forbidding any extra fields. When `strict` mode is requested, the serving infrastructure constrains the model's token-generation process itself (not merely a post-hoc validation check) so that only schema-conforming JSON can be produced. This is a materially stronger guarantee than the older prompt-engineering approach of simply *asking* a model to "respond in JSON" and hoping it complies, though as §6.2 discusses, this project's design does not fully trust that guarantee in practice, because strict schema support can vary across models routed through an aggregator like OpenRouter, hence the JSON-extraction fallback parser implemented defensively alongside it.

2.5 Rate Limiting Theory: The Token Bucket Model

API providers cap how many requests a client may make in a given time window to protect shared infrastructure. A **token bucket** is the standard conceptual model for enforcing such a cap: imagine a bucket that holds tokens, refilled at a constant rate (here, an allowance replenishing toward *R* requests per minute); each outgoing request consumes one token; a request is only sent once a token is available, and if the bucket is empty the caller waits.

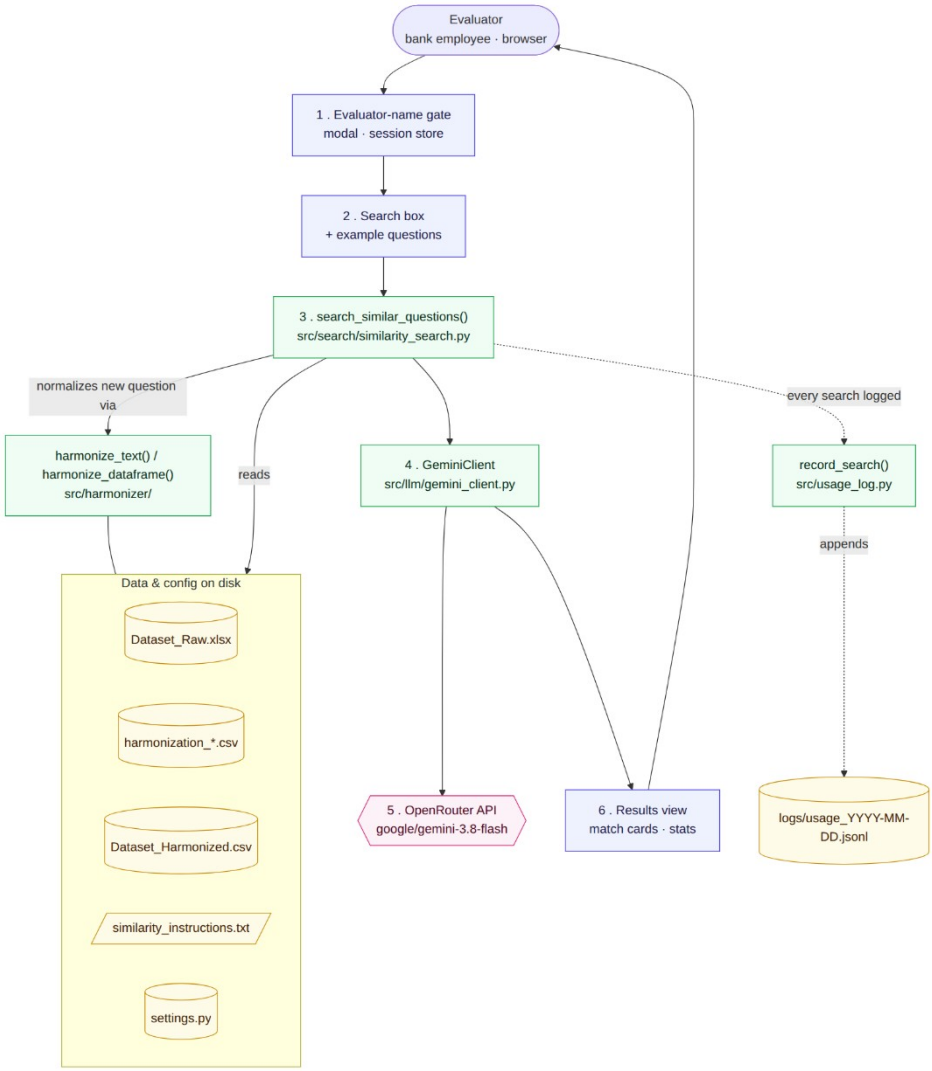
This project implements a simplified, single-token variant of that idea (see `RateLimiter` in `src/llm/gemini_client.py`, detailed in §6.3): rather than tracking a pool of accumulated tokens, it enforces a fixed **minimum interval** between consecutive calls, computed as `60.0 / RPM` seconds. Before every call, it measures elapsed time since the last call and sleeps for the remaining shortfall if the interval hasn't yet passed. This achieves the same steady-state throughput cap as a token bucket (no more than `RPM` requests per minute, on average) with a much simpler implementation, at the cost of not permitting short bursts above the steady rate the way a bucket with accumulated capacity would. Given this system serves one interactive evaluator issuing one search at a time (not a high-concurrency production service), that tradeoff is appropriate: predictable, gentle pacing matters more here than burst capacity.

2.6 Exponential Backoff with Jitter

When a request nonetheless fails due to rate limiting or a transient server error, simply retrying immediately tends to make the problem worse, especially under shared load, where many clients retrying in lockstep can produce synchronized request spikes (the "thundering herd" problem). The standard mitigation is **exponential backoff**: each successive retry waits a delay that grows exponentially with the attempt number (`2^attempt` seconds), giving the underlying service progressively more room to recover. **Jitter**, adding a small random offset to each computed delay for further desynchronizes retries from multiple independent clients that might otherwise back off in an identical, still-synchronized pattern.

This project's `GeminiClient.find_similar_questions()` (§6.3) implements exactly this pattern for rate-limit-class errors (detected by matching `429`, `rate`, or `resource_exhausted` in the exception text): `delay = (2 ** attempt) + random.uniform(0, 1)` seconds, up to `GEMINI_MAX_RETRIES` (5) attempts, after which the failure is surfaced rather than retried indefinitely, an important design discipline, since an unbounded retry loop against a persistently failing dependency is itself a reliability hazard (it can hang the calling UI indefinitely rather than surfacing an actionable error).

3. System Architecture



System architecture, as documented in the project's own README.md, showing the full request flow from evaluator through the harmonizer, GeminiClient, and OpenRouter, to the results view and the audit log.

3.1 End-to-End Pipeline

The system has two distinct execution paths that share the same harmonization core: an **offline/on-demand batch path** that harmonizes the raw dataset, and a **live search path** that runs once per evaluator query.

Batch path (`src/harmonizer/__init__.py: run_harmonization_raw()`):

1. Load `data/Dataset_Raw.xlsx` (the raw conversation log: `Conversation_ID`, `Date`, `Type`, `Conversation`).
2. Load both harmonization dictionaries (`data/harmonization_entities.csv`, `data/harmonization_time_formats.csv`) and compile them into regex patterns.
3. Clean Excel's datetime artifacts from the `Date` column; deterministically assign a `Question_Number` (Q1, Q2, Q3…) to every user row via `assign_question_numbers()`.

4. Harmonize the `Conversation` column, writing the result alongside the original as `Conversation_harmonized`.
5. Write `outputs/Dataset_Harmonized.csv` and a plain-text QA report (`outputs/dataset_harmonization_qa_report.txt`) summarizing every substitution made.

Live search path (`src/search/similarity_search.py: search_similar_questions()`), triggered once per evaluator query from the UI:

1. Re-run the batch path above in full, so any edits to `Dataset_Raw.xlsx` are always reflected without restarting the app.
2. Load the instructions file (`instructions/similarity_instructions.txt`) and the freshly harmonized dataset.
3. Normalize the evaluator's new question through the *same* compiled harmonization patterns used on the dataset.
4. Format every "user"-type row of the harmonized dataset as a single block of labelled plain text (`[ConversationID QuestionNumber] question text`).
5. Send the new question plus the full formatted dataset, in one call, to `GeminiClient.find_similar_questions()`.
6. Clean, re-key against the raw (non-harmonized) text for display, sort by score descending, and cap at `MAX_RESULTS_DISPLAYED` (20).
7. Render the results as match cards in the Dash UI.

3.2 Component Responsibilities

Module	Responsibility
`config/settings.py`	Single source of truth for every file path, column name, and tunable constant (model name, RPM cap, retry count, result cap). Nothing elsewhere in the codebase hardcodes these.
`src/harmonizer/harmonizer.py`	Pure-function library: load dictionaries, build the alias map, compile regex patterns, apply harmonization to a string or a DataFrame, write the QA report. No knowledge of project file layout.
`src/harmonizer/__init__.py`	The "application" layer over the harmonizer library: knows the real file paths, orchestrates the four-step batch pipeline, and owns the deterministic `Question_Number` assignment algorithm.
`src/llm/gemini_client.py`	The only module that talks to the model API. Wraps OpenRouter's OpenAI-compatible endpoint, defines the structured-output schema, and implements rate limiting and retry/backoff.
`src/search/similarity_search.py`	The live-search orchestrator: loads data, normalizes the query, builds the prompt payload, calls the LLM client, and defensively cleans the

Module	Responsibility
	returned matches for display.
app.py	UI only. Renders the Dash + Mantine interface and wires user interaction (typing, clicking examples, submitting a search) to search_similar_questions(). Contains no matching or harmonization logic of its own.

3.3 Module Call Graph

app.py depends on src/search/similarity_search.py, which depends on both src/harmonizer/harmonizer.py (for query-time normalization) and src/llm/gemini_client.py (for the actual model call); src/harmonizer/__init__.py sits alongside as the batch-mode entry point that similarity_search.py also calls at the start of every search to keep the harmonized CSV fresh. Every module, at every layer, reads its configuration from config/settings.py rather than from local constants, a single-source-of-truth design elaborated in Section 8.1.

A defining architectural principle, stated explicitly in the project's own README.md, governs the split of responsibility between the deterministic code and the model: **"Gemini does all the interpretation. The code does none of it."** The code's job ends at formatting the dataset as clean, labelled plain text; recognizing patterns *within* that data, including which rows are genuinely similar, and which pairs of consecutive rows represent one split question is left entirely to the model, guided by natural-language instructions rather than hardcoded heuristics. The one deliberate exception to this principle is Question_Number assignment (§4.4), which was moved *out* of the model's judgment and into deterministic code specifically because leaving it to the model proved unreliable.

4. Data Harmonization Module

Implemented in src/harmonizer/harmonizer.py (the pure-function library) and src/harmonizer/__init__.py (the application wrapper). This module is the practical realization of the canonicalization theory in §2.1.

4.1 The Harmonization Dictionaries

Two CSV files define the entire vocabulary: data/harmonization_entities.csv (financial terms, divisions, regions, actions, regulators, rating agencies, companies, strategies, and executives: 310 financial-term rows, 44 division rows, 18 region rows, 17 action rows, 14 regulator rows, 9 rating-agency rows, 4 company rows, 4 strategy rows, and 3 executive rows, 423 rows in total) and data/harmonization_time_formats.csv (months, quarters, quarter-years, half-years, full years, relative time expressions, and number-format conventions: 204 rows across 11 types). Each row has a Type, a Key (an internal identifier), a Canonical form, and up to seventeen Alias columns. For example, the DIVISION row for Corporate Banking lists Canonical = “Corporate Banking” with aliases CB, Corp Bank, corproate bank, Corporate Bank, corp banking, corporate division, corporate bnak, corp bnak, corportae banking, total corporate bank, corporate banking division, corporate clients; note the dictionary deliberately includes several misspellings of the same word (“corproate bank,” “corporate bnak,” “corp bnak,” “corportae banking”) because that is how it actually appears in evaluator-written text.

As of the final submitted version, the harmonization dictionary comprises two CSV files: harmonization_entities.csv (423 rows covering financial metrics, division names, companies, regulators,

rating agencies, regions, executives, strategies, and action verbs) and harmonization_time_formats.csv (204 rows covering quarters, half-years, full years, nine-month periods, months, and relative time expressions). Together these compile into 3,869 unique regex patterns. Over the course of the project dataset (637 rows total, 334 user rows), the harmonizer makes approximately 4,296 substitutions per run, resolving 244 distinct alias forms to their canonical equivalents. The QA report written to outputs/dataset_harmonization_qa_report.txt after each run lists every substitution made, sorted by frequency, providing a human-readable audit trail of exactly what the harmonizer changed and by how much.

A deliberate design principle governs alias coverage: every alias is financially accurate and domain-specific. Generic or ambiguous terms that could map to more than one canonical concept are excluded from the dictionary even when they appear frequently in the data, and misleading mappings identified during testing (for example, “NII sensitivity” incorrectly aliased to “Interest Rate Risk in the Banking Book”, or “tier 1 capital” aliased to “CET1 Ratio” despite these being distinct regulatory concepts) are corrected rather than left in place for coverage. The dictionary is therefore deliberately precise rather than maximally broad: an alias not enumerated falls through to the LLM’s own unaided semantic judgment, which is the correct fallback for genuinely ambiguous or unknown phrasing.

One category, NUMBER_FORMAT, is loaded alongside the others but explicitly excluded from substitution via HARM_SKIP_TYPES in config/settings.py: these rows describe formatting conventions (units like millions/billions/basis points), which are not meant to be found-and-replaced as free text the way a division or metric name is.

4.2 Building the Alias Map, Greedy Longest-Match

build_alias_map() flattens every dictionary row's alias columns into a single alias → canonical dictionary. Two correctness-critical rules govern this step:

1. **Case-insensitive collision handling.** All aliases are lowercased for lookup. An alias identical to its own canonical form is dropped (no point replacing "Revenue" with "Revenue").
2. **Longest-alias-wins.** Before deduplication, all (alias, canonical) pairs are sorted by alias length, descending, and only the *first* occurrence of each lowercased alias is kept. This is the **greedy longest-match** principle the same idea used in real tokenizers applied because some aliases genuinely overlap ("CB" is a short, generic-looking alias, while a longer, more specific phrase could exist for a different canonical term); processing longest-first guarantees the more specific mapping is never silently shadowed by a shorter, more generic one that happens to be iterated first.

4.3 Pattern Compilation and Text Substitution

_make_pattern() compiles each alias into a case-insensitive, word-boundary-anchored regular expression: \b + re.escape(alias) + \b. Two details matter here, and both are directly defensive against real failure modes:

- re.escape() neutralizes any regex metacharacters that happen to occur inside an alias (parentheses, slashes, periods, e.g. "C/I ratio" or "1,000k" appear verbatim in the dictionaries), so an alias is always treated as literal text, never as regex syntax.
- Word boundaries (\b) prevent a short alias like "IB" from matching inside an unrelated word such as "exhIBit", a real risk given how many two- and three-letter banking abbreviations exist in the dictionaries (RB, CB, IB, AM, PB, PM, WM…).

harmonize_text() then applies the compiled patterns to a string in four passes: (1) find every match from every pattern anywhere in the text; (2) discard any match that would create a **self-duplication** (see §4.3.1

below); (3) sort remaining matches by start position, breaking ties in favor of the *longest* match at that position (the same longest-match principle from §4.2, now applied at the text level, if "Corporate Bank" and "Corporate" both match at the same starting position, the fuller match wins); (4) walk the sorted matches left to right, keeping only non-overlapping ones. Substitutions are then applied to the string **right-to-left** (in reverse position order), so that editing a later part of the string never invalidates the character offsets of an earlier match still waiting to be applied.

4.3.1 The Self-Duplication Guard

A subtle bug class this design specifically defends against: an alias that is itself a prefix of its own canonical form. For example, "Retail" is a valid alias whose canonical is "Retail Banking", but if the source text already reads "Retail Banking", a naïve substitution of the word "Retail" alone would produce "Retail Banking Banking". `_would_duplicate_canonical()` checks, before accepting any such match, whether the text immediately following the match already continues with the remainder of the canonical phrase, and if so discards that match rather than corrupting already-correct text. This is a landmine that is easy to miss in testing (harmless on short-form input, silently corrupting on long-form input) and the project's own code comments note it was addressed deliberately rather than found by accident.

4.4 Deterministic Question_Number Assignment

`assign_question_numbers()` (in `src/harmonizer/__init__.py`) labels every user-authored row in the dataset with a `Q1`/`Q2`/`Q3`… tag, purely from row order within a conversation, no model call involved. The rule: the counter increments each time a new "user" row follows a non-user (model) row; but two **consecutive** user rows sharing the same `Conversation_ID` are treated as a **split question**, a short follow-up sent before the model had replied to the first part, and both rows receive the *same* Question_Number rather than incrementing.

This is explicitly called out in the code's own docstring as a deliberate correction to an earlier design: previously, the model itself was asked to infer question boundaries from row position, which the project notes was "correct most of the time but unreliable across chunk boundaries." Making this assignment fully deterministic, before the data ever reaches the model, removes an entire category of retrieval error, a mislabeled question boundary corrupting every match built on top of it, and is a direct, concrete illustration of the architectural principle from §3.3: interpretation is left to the model everywhere *except* where determinism can be guaranteed and materially improves reliability.

4.5 QA Reporting and Empirical Results

`write_qa_report()` writes a plain-text summary of every substitution applied, sorted by frequency, so a reviewer can sanity-check the dictionaries without diffing raw and harmonized CSVs by hand. On the final submitted dataset (637 rows, outputs/Dataset_Harmonized.csv), the harmonizer applied 4,296 total substitutions across 244 unique alias→canonical mappings. The most frequent substitutions illustrate exactly the abbreviation variance described in §1.1: roe → Return on Equity (323 occurrences), cir → Cost-Income Ratio (263), transaction banking → Transaction Banking (143), asset management → Asset Management (137), and change → Change Analysis (135), with Return on Equity alone being reachable through at least four distinct aliases in the observed data (roe, equity return, shareholder return, return on shareholders equity, return for shareholders).

A concrete before/after example from the harmonized dataset:

> ***Raw:*** *"Quick side-by-side on C/I ratio and equity return across corp banking, trade finance and Retail Bank for Q3_2025, which division is most cost-efficient, and does it hold up on ROE?"*
>
> ***Harmonized:*** *"Quick side-by-side on Cost-Income Ratio and Return on Equity across Corporate Banking, Transaction Banking and Retail Banking for Q3 2025, which division is most cost-efficient, and does it hold up on Return on Equity?"*

This single example demonstrates division aliasing ("corp banking" → "Corporate Banking", "trade finance" → "Transaction Banking", "Retail Bank" → "Retail Banking"), metric aliasing ("C/I ratio" → "Cost-Income Ratio", both "equity return" and "ROE" → "Return on Equity"), and time-period normalization ("Q3_2025" → "Q3 2025") operating together in one pass, exactly the compound normalization Section 2.1's theory describes as necessary before semantic comparison is meaningful.

5. Similarity Search Engine

Implemented in `src/search/similarity_search.py`. This module owns the live search path (§3.1) and is the point where the theory of §2.3 (LLM-as-judge retrieval) becomes a concrete prompt-construction and data-shaping problem.

5.1 Prompt Construction and the No-Merging Principle

`load_harmonized_rows()` reads the harmonized CSV exactly as-is: every row, original order, no merging or filtering beyond what the harmonizer already did, because recognizing which rows belong together (e.g. a split question spanning two rows) is deliberately left to the model, not to this function.

`format_dataset_for_prompt()` converts the DataFrame into a flat list of plain-text lines, one per "user"-type row, in the exact format `[ConversationID QuestionNumber] question text` (e.g. `[C42 Q3] Can you compare Revenue for...`), falling back to the raw `Conversation` column if the harmonized column is blank for a given row. Model-authored rows (answers, not questions) are skipped entirely, the model only ever needs to search over *questions*. This label format is what makes the `[ConversationID QuestionNumber]` references in the instructions file (§5.2) concrete and directly readable off each row, rather than requiring the model to infer structure from row position.

5.2 Prompt Design: The Instructions File

The full matching and scoring rubric lives in `instructions/similarity_instructions.txt`, sent as the system message on every call (see §6.1), not hardcoded in Python, so the matching behavior itself can be tuned by editing plain text with no code change. Its key design elements:

- **Data structure explanation**, teaching the model to read the `[ConversationID QuestionNumber]` label format and to treat two consecutive rows sharing an identical label as one combined split question (§4.4), rather than as two independent candidates.
- **Independence of co-located questions**: a single conversation contains three separate, unrelated questions (Q1–Q3); the instructions explicitly warn the model not to let a strong or weak match on one question from a conversation bias its judgment of a different, unrelated question from that same conversation.
- **Matching criteria**, defined as a compound judgment across four factors: the same division(s), the same metric(s), the same time period, and the same underlying ask (ranking, comparison, or flag for inconsistency), the direct operationalization of §2.3's argument for why an LLM's structured, multi-factor reasoning outperforms a single embedding-distance score for this task.

- **Continuous scoring**, explicitly instructing the model *against* defaulting to round "tier" values (0.90, 0.95, 0.98) even when several matches belong in the same broad quality band, with descriptive score-band guidance as a floor, not a bucket: 0.95–1.0 (essentially perfect on every factor), 0.85–0.94 (all factors match, minor imprecision in one), 0.70–0.84 (same core ask, one factor noticeably different), below 0.70 excluded entirely.

Output-equivalence reasoning: rather than scoring surface word overlap, the instructions direct the model to imagine the historical question's answer already produced and ask whether it would fully and directly satisfy the new question. Concrete worked examples are given for why a seemingly small divergence can still break equivalence: an extra quarter at the end of a trend is not a cosmetically longer period, because Q4 carries year-end provisioning, write-offs, and management overlays that can flip the reported direction; Cost of Risk and ECL coverage are not interchangeable credit-health metrics, since a division with a low CoR can still show deteriorating ECL coverage, and the instructions explicitly warn that when more than one factor diverges at once (metric and period, or scope and ask), the gaps compound rather than average out, and the score should fall correspondingly more sharply than for a single difference.

Exhaustive-search requirement: the instructions state plainly that the dataset was deliberately constructed with roughly 270 unique questions across 91 conversations, many repeated in different phrasings, and that finding one strong match is not a signal to stop: the model is required to scan to the end of the dataset before finalizing its answer, since missing a second or third genuine match by stopping early is treated as a real failure mode, not an edge case. This 270/91 figure is now stale: the dataset has since grown further, to 334 unique questions across 116 conversations (§4.5) without `instructions/similarity_instructions.txt` being refreshed to match. This does not change the model's behaviour (the exhaustive-scan instruction is qualitative, not tied to the literal count), but the stale number is worth correcting the next time this file is edited, so the size the model is told to expect matches the size it is actually asked to search.

5.3 The No-Chunking Design Decision

An earlier version of this system split the dataset into smaller batches and issued one LLM call per chunk (`SEARCH_CHUNK_SIZE`, referenced in `config/settings.py`'s own comments as a now-removed setting). The final, submitted design sends the **entire** dataset in a **single** call per search, a deliberate client-requested change, not an oversight. This tradeoff is stated explicitly in both the module docstring and `config/settings.py`: single-call search is cheaper (one API request instead of several against a constrained quota) but was empirically found *less* reliable at exhaustively checking every candidate row than the earlier chunked approach. The §5.2 exhaustive-search instruction exists specifically to mitigate the reliability cost of this tradeoff by compensating at the prompt level for what chunking used to guarantee structurally (bounding how much the model has to scan in any one call).

5.4 Raw-vs-Harmonized Text Separation

A structural requirement threading through this module: the model must only ever be shown **harmonized** text (so its matching is reliable and vocabulary-consistent), but the evaluator viewing results must see the **original, raw** text they or a colleague actually typed: harmonized text like "Cost-Income Ratio" would look artificial and unfamiliar in a UI meant to feel like plain conversation review.

`build_raw_question_lookup()` solves this by building a dictionary keyed on `(conversation_id, question_number)` not on matching text, which the earlier design relied on and which could silently fail whenever the model paraphrased its echoed text even slightly. Because the structured-output schema (§6.2) requires the model to report `conversation_id` and `question_number` directly, this lookup only needs an exact key match. Two additional defensive details: (1) the key is **case-normalized to uppercase** on both sides, because the

model has been observed echoing conversation IDs back in a different case than the source data (e.g. c91 for the real ID C91) a case-sensitive key here previously caused a silent lookup miss that fell back to the model's own (sometimes imprecise) matched_question_text for an otherwise correctly-scored match; (2) for a split question (two rows sharing one key), the **longer** of the two raw texts is kept as the representative one, so a short follow-up fragment indexed first doesn't win over the fuller main question.

5.5 Result Cleanup, Sorting, and Capping

search_similar_questions() defensively filters the model's raw returned matches, keeping only entries with a valid conversation_id and a numeric similarity_score (silently dropping anything malformed rather than crashing the whole search on one bad entry, the same skip-and-warn philosophy seen throughout the harmonizer). Each surviving match is rounded to three decimal places, given a display percentage string, re-keyed to the raw text via §5.4's lookup, sorted by score descending, and truncated to MAX_RESULTS_DISPLAYED (20) results before being handed to the UI.

6. LLM Integration Layer

Implemented in src/llm/gemini_client.py, the single module permitted to talk to the model API. Every other module reaches the model only through GeminiClient.find_similar_questions().

6.1 Routing Through OpenRouter

The model used is Google's google/gemini-3.8-flash, but it is **not** called via Google's native google-genai SDK. Instead, the client talks to OpenRouter (https://openrouter.ai/api/v1), which exposes an OpenAI-compatible /chat/completions endpoint in front of many providers' models, using the standard openai Python package pointed at OpenRouter's base URL. This is a meaningful design choice with a direct practical benefit: because the public interface of GeminiClient (a find_similar_questions() method) is unchanged from an earlier, Google-SDK-native version, no other module needed to change when the underlying transport swapped, similarity_search.py still just calls client.find_similar_questions(...).

One naming quirk worth documenting precisely: GEMINI_API_KEY in .env and config/settings.py is, despite its name, actually an **OpenRouter** API key, kept under the original name deliberately to minimize disruption to .env, the README, and every other place that name already appears in the project.

6.2 Structured Output via response_format

Because OpenRouter's endpoint is OpenAI-compatible rather than Gemini-native, structured output is requested through OpenAI's response_format parameter (type: "json_schema", strict: true) rather than Gemini's native response_json_schema the concrete API-level realization of the theory in §2.4. The schema (SIMILARITY_RESPONSE_SCHEMA) defines a top-level object with one matches array; each element is an object with five **required** fields conversation_id, question_number, matched_question_text (the exact verbatim matched text, so the row can be traced), similarity_score, and reasoning and additionalProperties: false throughout, since OpenAI's structured-output implementation requires every property to be explicitly listed and no extras permitted.

The fallback parser. Whether a given model routed through an aggregator like OpenRouter honors response_format with full strictness can vary, so _parse_response() does not blindly trust a clean parse. It first attempts a direct json.loads() of the raw response; if that fails, it searches the text for the first {...} block via regex and attempts to parse that instead (handling a reply that arrived with stray prose wrapped around otherwise-valid JSON). If neither succeeds, it checks for a specific, diagnosable failure signature: text that *starts* like valid JSON ({ followed by "matches" within the first 50 characters) but never closes

with a matching `}` this pattern indicates **truncation**, i.e. the model ran out of its output-token budget mid-reply, and the resulting `ValueError` says so explicitly and points the developer at raising `MAX_OUTPUT_TOKENS`, rather than leaving them to misdiagnose a token-budget problem as a prompt or schema bug.

6.3 Rate Limiting and Retry Logic

`RateLimiter` implements the simplified token-bucket variant described theoretically in §2.5: before every call, `wait()` measures elapsed time since the previous call and sleeps for any shortfall below `60.0 / GEMINI_RPM` seconds, self-throttling to a conservative 10 requests per minute, kept comfortably under the account's real limit so a burst of interactive use from the UI never trips a 429.

`find_similar_questions()` wraps every call in a retry loop of up to `GEMINI_MAX_RETRIES` (5) attempts. On any exception whose text matches `429`, `rate`, or `resource_exhausted` (case-insensitive), it applies the exponential-backoff-with-jitter formula from §2.6 `(2 ** attempt) + random.uniform(0, 1)` seconds before retrying; any other exception triggers a flat one-second pause before the next attempt. If every attempt is exhausted, a `RuntimeError` wrapping the last underlying error is raised rather than failing silently, which `similarity_search.py` and, ultimately, the UI's `friendly_error()` translator (§8.3) surface to the evaluator in plain language.

6.4 The max_tokens Ceiling

`MAX_OUTPUT_TOKENS` is set explicitly to 20,000 on every call. This is documented in the code as a genuine correctness fix, not merely a workaround: without an explicit cap, the client library requests the model's full default maximum output (65,536 tokens for `gemini-3.8-flash`), and OpenRouter prices and approves the request against that worst-case ceiling, which in practice produced a 402 "insufficient credits" error even though the real response (a JSON array of matches with reasoning text) is only ever a few thousand tokens, even for a search returning many genuine matches. The value was raised once already, from an initial 8,000, after a real search with a large number of genuine matches was cut off mid-JSON at that lower ceiling, exactly the truncation failure mode §6.2's fallback parser is designed to detect and diagnose.

6.5 Debug Visibility

Every successful call logs, to the console, the count of matches returned and, for each, its conversation ID, question number, score, and the first 120 characters of its matched text, captured *before* `similarity_search.py`'s downstream cleanup and filtering step (§5.5). This makes it possible to distinguish, during development or troubleshooting, whether an expected match was never considered by the model at all versus scored and then dropped somewhere further down the pipeline. When the model supplies a reasoning string for a given match, that reasoning is now logged immediately beneath it as well.

7. User Interface Design

Implemented in `app.py`, using Dash paired with Dash Mantine Components (DMC). This file is UI only: it contains no matching, scoring, or harmonization logic of its own, calling into `search_similar_questions()` for everything substantive. The interface was deliberately designed to be self-explanatory to a non-technical audience explicitly noted in the file's own docstring as suitable for client-management demonstrations.

7.1 Component Architecture

Dash is a reactive, callback-driven framework: the page is described once as a static Python object tree (app.layout), and subsequent updates happen by callback functions returning new values for specific named outputs, with no full-page reloads. The layout here is wrapped in dmc.MantineProvider (supplying a shared THEME an indigo primary color, consistent card/paper/badge styling, and the Inter font family) and built from composable builder functions: header(), how_it_works(), search_card(), legend(), stat_card(), match_card(), skeleton_results(), and empty_state() each a small, self-contained function returning a Mantine component tree, composed together in app.layout.

7.2 The Explainer and Input Surface

how_it_works() renders a three-step visual strip ("1. Ask → 2. Compare → 3. Review"), each step paired with a small inline base64-encoded SVG icon built by _svg_icon() (search, chat, database, and list glyphs generated inline rather than loaded as external image assets, keeping the page fully self-contained). search_card() provides the question textarea, three clickable pre-filled example questions (EXAMPLES, each a labeled button whose tooltip shows the full question text), a Clear button, and a Search button; app.index_string injects a small client-side script binding Ctrl/Cmd+Enter to trigger the search button, so power users are not forced to reach for the mouse.

7.3 Presenting Match Strength in Plain Language

A core design goal is that a non-technical reviewer should never have to interpret a bare decimal score. BANDS defines three labeled strength tiers mirroring the exact score-band language from the instructions file (§5.2) "Near-identical question" (≥95%), "Same question, minor differences" (≥85%), and "Closely related question" (≥70%), each with its own color and a plain-English description shown on hover. band(pct) maps a percentage to its label and color; legend() renders these as a small reference key above the results; match_card() renders each individual result as a card combining the raw matched question text, a dmc.RingProgress circular percentage gauge colored by band, and "why it matches" reasoning chips produced by split_reasons() a small parenthesis-depth-aware splitter that breaks the model's free-text reasoning (e.g. "same division (WM), same metrics (Cost and Revenue), same period") into short, individually readable chips without splitting inside parenthetical asides, falling back to showing the reasoning as one unsplit sentence if the text doesn't split cleanly (more than six pieces, or any piece implausibly long).

7.4 Loading, Empty, and Result States

Three distinct visual states are handled explicitly: skeleton_results() (animated placeholder cards with a "comparing your question…" message, shown while Dash's built-in running=[...] callback mechanism toggles button loading spinners and disables the Clear button mid-search), empty_state() (the initial dashed-border prompt shown before any search has been run), and the populated results view a headline stating the count of matches in plain language ("3 earlier questions ask the same thing"), four summary stat cards (matches found, best match, average match, time taken), the legend, and the ranked list of match cards. A distinct, separate "no matches found" state is rendered when a genuine search completes with zero results, worded to reassure the evaluator this is an expected outcome ("this looks like a question the system hasn't been asked before") rather than something that looks like a failure.

7.5 Friendly Error Translation

friendly_error() is a pattern-matching translation layer between raw exception text and audience-appropriate messaging: it lowercases the exception string and checks it against a sequence of substring

patterns (API-key problems, rate-limit/quota exhaustion, an unrecognized model name, a missing dataset or instructions file, a network/connectivity failure), returning, for each recognized case, a short plain-language headline, a slightly more detailed explanation, and the original raw exception text. The raw text is never discarded it is shown in a collapsible dmc. Accordion panel labeled "Technical details (for the team)," so a non-technical evaluator sees a calm, actionable message while a developer can still expand the same alert to see the exact underlying error. Any exception not matching a known pattern falls through to a generic "something went wrong while searching" message, so the UI never surfaces a raw, unstyled stack trace.

7.6 Additional Interaction Details

A dark-mode dmc.Switch toggles forceColorScheme on the Mantine provider; Dash notifications (dmc.NotificationContainer) confirm search outcomes (success, no-match, or error) as transient bottom-right toasts; and the _TrackedClient wrapper (§8.4) distinguishes a genuinely empty result set from a search where every underlying model call failed, two situations that would otherwise look identical to the UI (an empty list) but call for very different messaging to the evaluator.

8. Configuration and Reliability Engineering

8.1 Single Source of Truth

config/settings.py is the one place every file path, column name, and tunable constant in the project is defined: harmonization file paths and skip-types, the raw dataset path and output paths, the text columns to harmonize, the model name and API settings, the rate-limit and retry caps, the instructions-file path, and the results-display cap. Every other module imports these rather than hardcoding a path or a magic number locally. This is a deliberate maintainability choice, called out directly in the module's own docstring: when a client renames a column or moves a file (noted in comments as something that has genuinely happened during development), there is exactly one place in the codebase to fix it, rather than a search-and-replace across every module that touches that value. load_dotenv() is called at import time so environment-specific secrets (the API key) are read from a local .env file, kept out of source control, while every non-secret setting stays as an explicit, readable Python constant.

8.2 Environment and Secrets Management

The python-dotenv package loads .env at startup; GEMINI_API_KEY (in practice an OpenRouter key, §6.1) is read via os.getenv(), so no credential is ever committed to source. GEMINI_MODEL, GEMINI_RPM, and GEMINI_MAX_RETRIES are all adjustable as plain constants without touching any calling code, and app.py similarly reads DASH_HOST, DASH_PORT, DASH_DEBUG, and DASH_NO_BROWSER from the environment for deployment-time flexibility (binding address/port, hiding or showing Dash's developer toolbar, and suppressing the auto-open-browser convenience during headless or automated runs).

8.3 Defensive, Graceful-Degradation Design

A recurring philosophy runs through every layer of this codebase: prefer skipping and logging a warning over crashing the whole run on one bad input. Concrete instances: harmonize_dataframe() warns and skips a requested column that doesn't exist in a given DataFrame rather than raising; compile_patterns() skips a single alias that somehow fails to compile into valid regex rather than aborting pattern compilation entirely; run_harmonization_raw() prints a clear SKIP message and still writes an (empty) QA report if the raw dataset file is missing, rather than letting one missing file take down a pipeline that could one day process several files in sequence; and search_similar_questions()'s result-cleanup step (§5.5) discards

individually malformed match entries from the model's response rather than failing the entire search over one bad record. Every Type-column comparison in both the harmonizer and the search module normalizes with .strip().lower() before matching, so a stray trailing space in a hand-edited spreadsheet: seven rows in the current data/Dataset_Raw.xlsx carry "user " or "model " instead of "user"/"model", does not silently drop rows from harmonization or search. SearchInputError, a small dedicated exception class, is raised deliberately at every upfront validation point (missing dataset, missing instructions file, empty dataset, missing required columns) *before* any API call is made, so a data problem is diagnosed immediately and cheaply, rather than surfacing confusingly after an expensive model call has already run.

8.4 Distinguishing "No Matches" from "Search Failed"

app.py's _TrackedClient wrapper (§7.6) addresses a specific ambiguity: both a genuinely novel question (correctly returning zero matches) and a search where every underlying model call errored out would otherwise produce the same empty list to the UI. _TrackedClient counts total calls made and records every exception raised; run_search_safely() then checks whether the number of recorded errors equals the number of calls attempted, and if so raises a RuntimeError carrying the last real error rather than allowing the caller to render a false "no earlier question asks the same thing" success state. This is a direct application of the same reliability principle as §8.3, applied specifically to prevent a failure from being silently misrepresented as a valid empty result.

8.5 The Friendly-Error Translation Layer

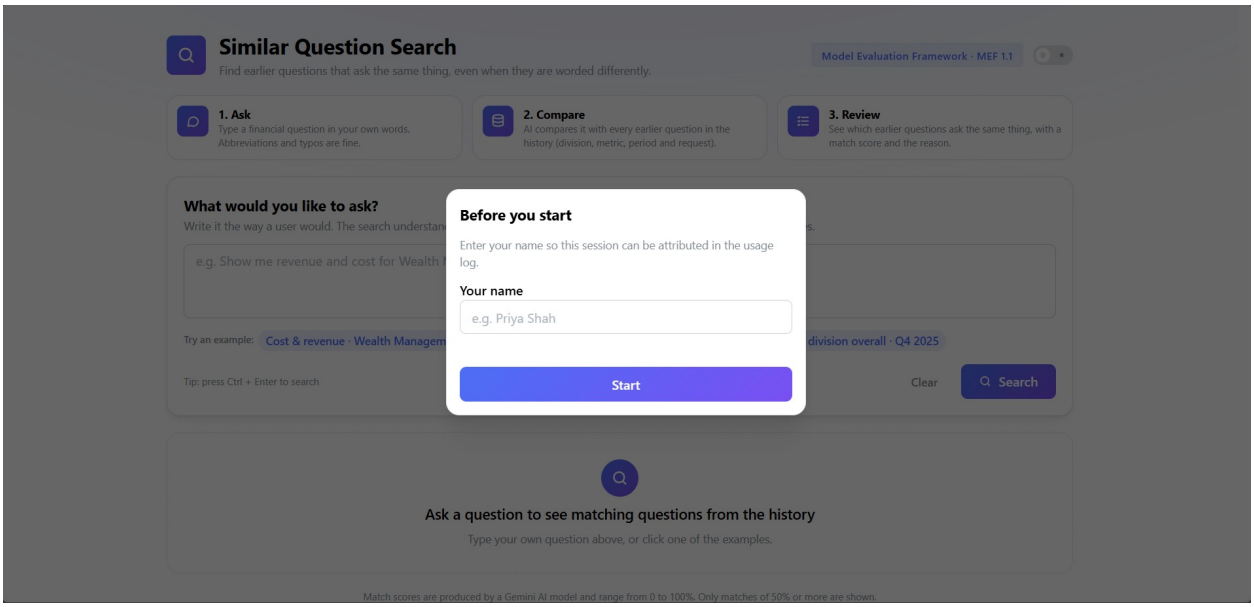
Detailed functionally in §7.5; architecturally, it is worth noting that this layer sits entirely in the UI module and never in the lower layers: similarity_search.py and gemini_client.py raise ordinary, technically precise exceptions (SearchInputError, ValueError, RuntimeError), and only app.py's friendly_error() translates those into audience-facing language at the last possible point before display. This separation keeps the lower layers' error messages precise and debuggable while still giving the UI full latitude to decide how much technical detail different audiences should see.

9. Results and Demonstration

This section walks through a single real interaction, captured directly from the deployed application (not staged or simulated), across all four states the UI actually passes through: the mandatory evaluator gate, the search box being filled and submitted, the loading state, and the populated results. The query used throughout is one of the tool’s own built-in example questions (§7.2), chosen because it stacks two different abbreviation styles (CB, HNW) with a compound, two-metric request into a single question, a reasonable stress test of both the harmonization layer and the LLM comparison step.

9.1 Step 1: The Evaluator Opens the Tool

Before any of that is usable, the app shows a blocking modal (evaluator_gate(), §7.1) prompting the evaluator to enter their name. There is no close button and no click-outside or Escape dismissal: typing a name and pressing Start (or Enter) is the only way through it. The name is stored in a session-scoped browser store and attached to every search that evaluator runs for the rest of that browser tab, so the audit log (§6, src/usage_log.py) can record who ran what, not just what was asked.

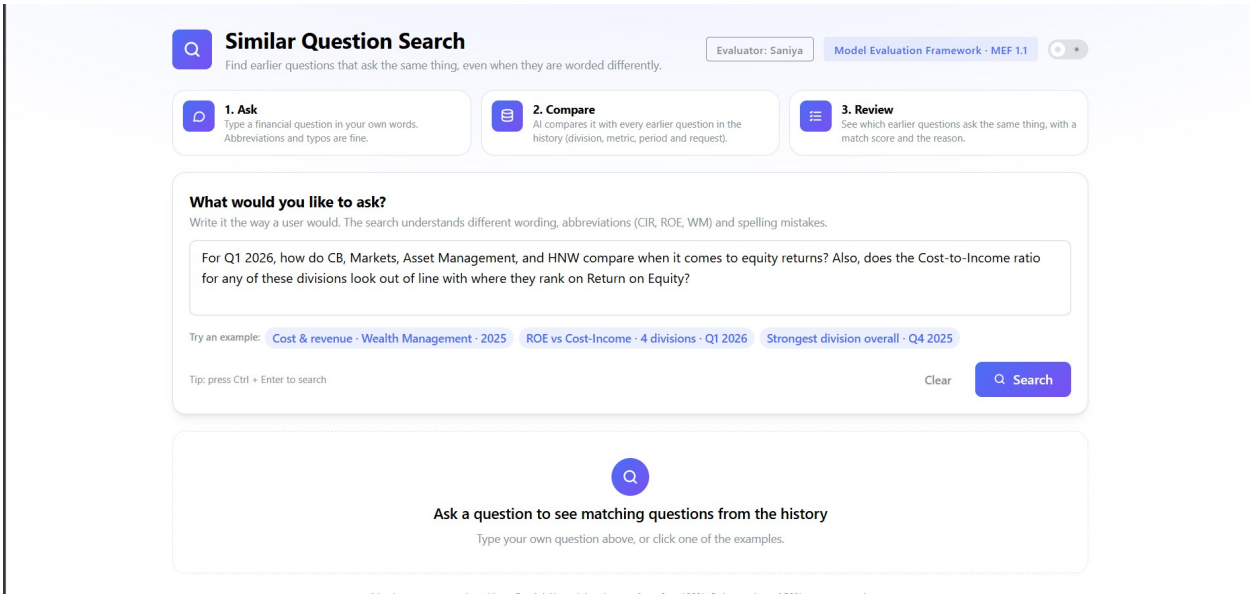


Once the evaluator’s name is submitted and the modal closes, they see the header, the three-step "How it works" explainer strip, and the empty search card with its textarea, three clickable example buttons, and the Search/Clear controls. No results are shown yet: `empty_state()` (§7.4) invites the evaluator to type a question or click an example.

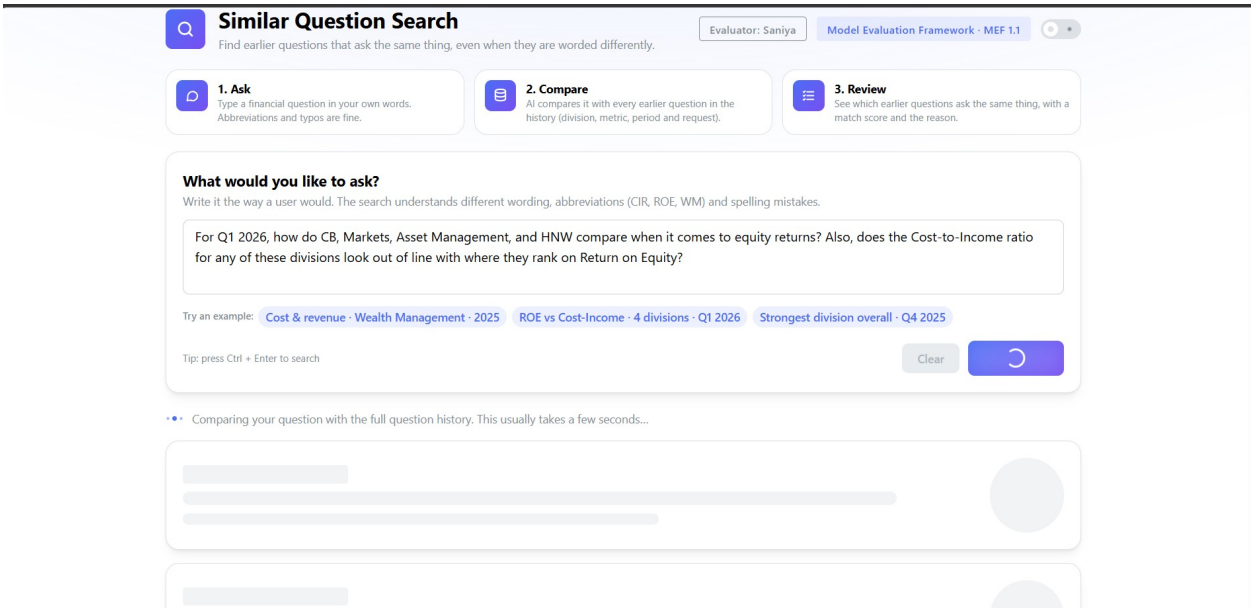
9.2 Step 2: Entering and Submitting a Question

The evaluator types a question directly into the search box, exactly as it occurs to them, with abbreviations, typos, and shorthand left in. On clicking Search (or pressing Ctrl/Cmd+Enter), Dash's running=[...] mechanism (§7.4) puts the Search button into a loading state, disables Clear, and swaps in skeleton_results(): animated placeholder cards with a “Comparing your question with the full question history…” message, while, behind the scenes, the full pipeline executes: the dataset is re-harmonized (§3.1), the new question is normalized through the same harmonization patterns (§4), the entire harmonized dataset is formatted and sent to Gemini in one call (§5, §6) under the rate limiter and retry logic (§6.3), and the returned matches are cleaned, re-keyed to raw text, sorted, and capped (§5.5).

The two screenshots below were captured from a live session run for this report, using the second built-in example question from `EXAMPLES` (§7.2), chosen because it stacks two separate abbreviation styles (CB, HNW) with a compound request (equity returns *and* Cost-to-Income ratio across four divisions) in a single question, exercising both the harmonizer and the LLM comparison step more heavily than a simple one-metric query would.



The filled search box immediately before Search is clicked. The evaluator badge (top right, "Evaluator") confirms the gate from §9.1 is already satisfied; the badge beside it ("Model Evaluation Framework · MEF 1.1") is the static version tag set in header() (§7.1).

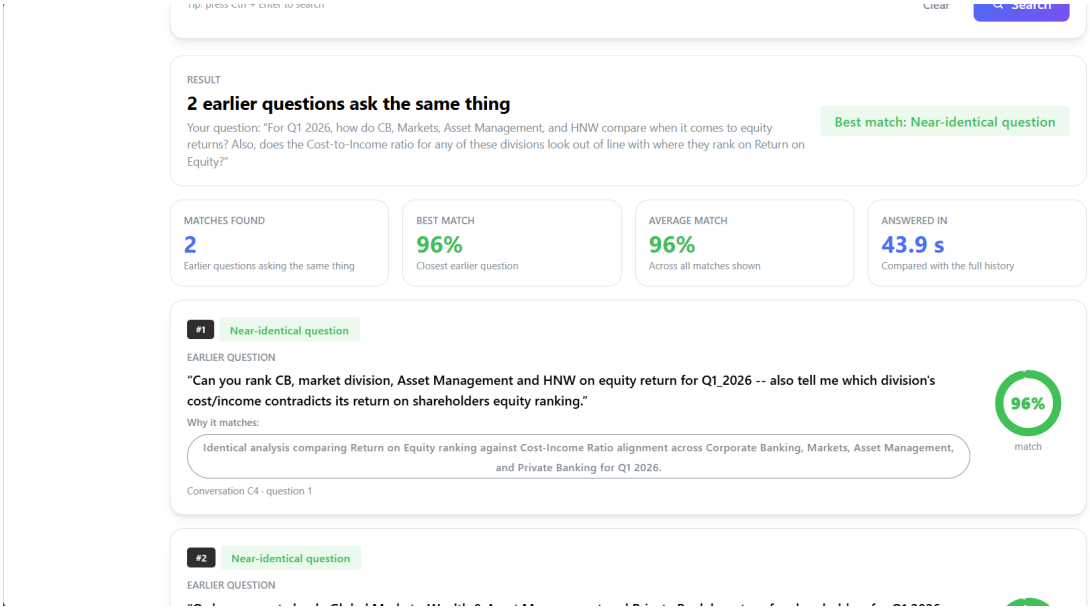


Immediately after Search is clicked: the button enters its loading state, Clear is disabled, and `skeleton_results()` (§7.4) renders animated placeholder cards with a "Comparing your question with the full question history…" message while, behind the scenes, the pipeline described in the paragraph above actually executes.

9.3 Step 3: Reviewing the Results

Once the search returns, the UI replaces the loading skeleton with the populated results view: a plain-language headline stating how many earlier questions ask the same thing, four summary stat cards (matches found, best match percentage, average match percentage, and time taken), the match-strength legend, and one ranked match card per result: each showing the raw historical question text, a colored ring-progress gauge with its match percentage, a plain-language strength label ("Near-identical

question” / “Same question, minor differences” / “Closely related question”), “why it matches” reasoning chips, and a tooltip identifying exactly which conversation and question number it came from.



The populated results view for the same query as §9.2, captured moments after the loading state.

The headline reports “2 earlier questions ask the same thing,” and the stat row shows a 96% best match, a 96% average across the matches shown, and a 43.9 s round trip (re-harmonization plus the single Gemini call, §5.3). The visible match card (#1, Conversation C4) is banded “Near-identical question” (§7.3) at 96%, showing its raw historical question text (“Can you rank CB, market division, Asset Management and HNW on equity return for Q1_2026, also tell me which division’s cost/income contradicts its return on shareholders equity ranking”), its “why it matches” reasoning chip naming the divisions and metrics that lined up, and a tooltip-backed caption identifying exactly which conversation and question number it came from (§5.4). A second match card begins immediately below it, cropped out of this particular capture. Repeated runs of the same query do not always return an identical set of matches, consistent with §2.3’s discussion of the LLM-as-judge paradigm (Gemini’s scoring is not perfectly deterministic run to run) and a useful reminder that the exhaustive-scan instruction (§5.2) is a strong request to the model, not a guarantee enforced by the code around it.

9.4 What This Walkthrough Demonstrates

Taken together, the four captures above demonstrate the complete round trip described architecturally in §3.1: an evaluator is identified before anything else runs, an informally-phrased question is silently normalized to the same canonical vocabulary as the historical log, judged for genuine semantic equivalence by the LLM rather than by surface text matching, and presented back to a non-technical reviewer in language that requires no understanding of similarity scores, regular expressions, or LLM prompting to interpret correctly.

10. Limitations and Future Work

10.1 Known Limitations

- **No-chunking reliability tradeoff.** As documented in the project's own code and comments (§5.3), sending the full dataset in a single call is cheaper but was found empirically less reliable at exhaustive candidate checking than the earlier chunked design. The exhaustive-search prompt instruction (§5.2) mitigates but does not structurally eliminate this risk: a model that scans hastily can still under-report matches in a way a hard chunk boundary would have prevented by construction.
- **Rate and cost ceiling.** The self-imposed 10 requests-per-minute cap (§6.3) is conservative by design, appropriate for a single interactive evaluator but a real constraint if this tool were extended to serve concurrent users or batch-mode evaluation runs.
- **No systematic ground-truth evaluation set.** Correctness has not been validated against a systematic, labeled evaluation set covering the full 0.0–1.0 score range, nor against a documented set of hand-verified ground-truth cases. Scoring consistency across borderline cases (§2.3's accepted non-determinism tradeoff) is not currently measured quantitatively.
- **Re-harmonization cost on every search.** `search_similar_questions()` re-runs the full harmonization pipeline (§3.1, step 1) on every single search so that edits to the raw dataset are always reflected, trading a small amount of redundant per-search computation for always-fresh data with no separate "rebuild" step to remember to run. This adds approximately 5 seconds to every search interaction.
- **Dictionary coverage is finite by design.** As discussed in §2.1, alias substitution only resolves phrasing explicitly enumerated in the dictionaries; wording outside that enumerated set falls through to the LLM's own (unaided) semantic judgment rather than being normalized first. Unknown abbreviations (such as RAROC, PCG, GTB, RBB) that appear in evaluator questions but are absent from the dictionary reach the model in their raw form, which the model may or may not resolve correctly based on general financial knowledge.
- **Scoring calibration on borderline cases.** Early versions of the instruction file used an explicit factor-based scoring rubric (same division ✓, same metric ✓, same period ✓, same ask ✓) that produced systematically over-confident scores on cases where one factor differed meaningfully: for example, a pair where one extra division in a ranking could change the winner consistently scored 0.90 rather than the more appropriate 0.70–0.75. The current instruction file replaces this with analyst-judgment framing centred on output interchangeability, which has reduced over-confident scores on the cases tested. However, calibration across the full score range remains informal rather than systematically measured.

10.2 Possible Extensions

- **Embedding-based pre-filtering.** A cheap first-pass embedding similarity search could shortlist candidates before the LLM-as-judge stage runs, combining the precision of §2.3's structured reasoning with the scalability of vector retrieval, relevant if the historical dataset grows well beyond its current ~334-question scale.
- **Result caching.** Repeated or near-identical queries could reuse a prior search's result rather than issuing a fresh model call every time.
- **Batch evaluation mode.** A mode to check an entire new set of proposed questions against the historical log in one run, rather than one evaluator-typed question at a time, would let the whole framework be used proactively during log curation rather than only reactively per question.

- **Quantitative scoring-consistency evaluation.** Building a larger, systematically labeled test set covering the full score range would let the continuous-scoring design (§5.2) be measured for score stability across repeated runs and for calibration accuracy across different score bands, not just top-line match/no-match correctness.
- **Harmonization dictionary expansion.** The current dictionary was built around the Deutsche Bank segment structure and a core set of financial metrics. Extending it to cover additional institutional contexts, product-level terminology (structured finance sub-products, prime brokerage metrics, ALM-specific measures), and emerging regulatory vocabulary (DORA, Basel IV output floor specifics) would reduce the proportion of evaluator phrasing that reaches the LLM unnormalised.
- **Dataset re-harmonization caching.** Rather than re-running the full four-step harmonization pipeline on every search, a file-modification-time check could detect whether either the raw dataset or the dictionary files have changed since the last harmonization run, and skip the pipeline if neither has. This would remove the ~5-second overhead from the majority of searches without sacrificing the always-fresh guarantee for the minority where the data actually changed.

11. Conclusion

The Similarity Scoring Engine delivers on its core requirement: given a new evaluator question, it reliably surfaces every historical question that genuinely asks the same thing, despite arbitrarily different phrasing, and does so in a form a non-technical reviewer can trust and act on without needing to understand the machinery underneath. Two design decisions define the system's character. First, splitting the problem into a deterministic **canonicalization** stage and a judgment-based **LLM-as-judge retrieval** stage means each stage is doing the work it is actually good at: code handles exact, rule-governed substitution reliably and cheaply, while the model handles the genuinely ambiguous, multi-factor semantic judgment that a fixed distance metric could not express as well. Second, a consistent architectural discipline, "the model does all the interpretation, the code does none of it," is applied everywhere except where determinism demonstrably improves reliability, most visibly in the deterministic Question_Number assignment (§4.4) that replaced an earlier, less reliable model-inferred version.

The instruction file that guides the model’s judgment went through meaningful iterative refinement during development. Early versions specified explicit scoring criteria tied to named factors (division match, metric match, period match, ask match), which produced a mechanical checklist effect: the model accumulated small per-factor deductions rather than reasoning about whether the outputs of two analyses would actually be interchangeable. The current version replaces this with analyst-judgment framing: asking the model to reason as a senior financial analyst asking whether it would produce the same analysis for both questions, which has meaningfully improved score calibration on borderline cases.

Beyond the working tool itself, this project is a concrete, hands-on demonstration of the same LLM-as-judge evaluation pattern that the broader Model Performance Evaluation Framework studies: a model reasoning over structured criteria to produce a scored, justified judgment, with every design choice (canonicalization, prompt design, structured output, rate limiting, defensive error handling) made explicit and documented rather than left implicit. For the Deutsche Bank collaboration context this project is framed around, that transparency is the point: an evaluation tool is only as trustworthy as an institution’s ability to understand exactly how it reaches its judgments, and this report has aimed to make that understanding complete, from the underlying theory down to the specific line of code implementing it.

The result is a system that gives evaluators an immediate, reliable answer to the question they face every time they sit down to test the model: has this been asked before?

Section 12 turns that documentation into a runnable artifact: a reader with no prior exposure to this codebase can follow it, unaided, from a fresh clone to a working local instance, or through to a deployed one on the university Data Lab.

12. Implementation and Deployment Guide

The preceding sections explain what the Similarity Scoring Engine does and why it is built the way it is; this section is the complementary how-to-run-it guide. It is written to stand on its own: a reader can follow it directly, without having read the rest of the report first, to install the project locally, confirm it works, or deploy it to the university Data Lab. Every command, file path, and configuration value below is taken directly from the project's own `README.md`, `requirements.txt`, `pyproject.toml`, and `config/settings.py`, not paraphrased, so it can be followed exactly as written against the actual repository.

12.1 Prerequisites

- **Python 3.11 or later.** Check with `python --version`.
- **Git,** to clone the repository.
- **An OpenRouter API key** (openrouter.ai/keys). The project's `GEMINI_API_KEY` environment variable is, despite its name, an OpenRouter key rather than a Google AI Studio key, because the model (`google/gemini-3.8-flash`) is routed through OpenRouter's OpenAI-compatible API rather than called directly (§6.1); see `src/llm/gemini_client.py`'s own module docstring for why.
- **No GPU or paid infrastructure** is required for local development: the app does no local model inference; it only calls out to OpenRouter.
- **For deployment rather than local use:** access to the university's Portainer instance on the Data Lab host, and a GitHub personal access token with the `read:packages` scope (§12.6).

12.2 Local Installation (Quick Start)

Clone the repository and install it as an editable package with its development extras, which also pulls in the testing tools used in §12.5:

```
git clone https://github.com/SRH-Heidelberg-University-ADSA/
SimilaritySearch.git
cd SimilaritySearch
pip install -e ".[dev]"
cp .env.example .env
# edit .env and set GEMINI_API_KEY to your real OpenRouter key
python app.py
```

Then open http://127.0.0.1:8050 in a browser, enter a name at the evaluator-gate prompt (§9.1), and either type a question or click one of the built-in example buttons (§7.2).

A plain runtime install without the dev/test tooling is also possible, by substituting `pip install -r requirements.txt` for the `pip install -e ".[dev]"` line above; `requirements.txt` is deliberately kept in sync with `pyproject.toml`'s dependency list for exactly this case.

12.3 Configuration Reference

Every tunable value the application reads from the environment is listed below. config/settings.py (§8.1) is the single source of truth these are loaded into, so nothing else in the codebase should hardcode a path, column name, or magic number that belongs here instead.

Variable	Required	Default	Meaning
GEMINI_API_KEY	Yes	none	Your OpenRouter API key (see §12.1's note on the name).
DASH_HOST	No	127.0.0.1	Bind address. The Dockerfile sets this to 0.0.0.0 for containers.
DASH_PORT	No	8050	Port the Dash server listens on.
DASH_DEBUG	No	0	Set to 1 to show Dash's developer toolbar.
DASH_NO_BROWSER	No	0	Set to 1 to skip auto-opening a browser tab (used in Docker).
LOG_LEVEL	No	INFO	DEBUG / INFO / WARNING / ERROR / CRITICAL.

12.4 Running the Harmonization Pipeline Independently

A separate harmonization run is not normally necessary: app.py runs it automatically on first startup if outputs/Dataset_Harmonized.csv does not yet exist, and search_similar_questions() (§5.1) re-runs it before every single search, so edits to data/Dataset_Raw.xlsx or to the harmonization dictionaries (§4.1) take effect immediately, without a restart. To run it manually regardless, for instance, to inspect the QA report (§4.5) without starting the UI at all:

```
python -c "from src.harmonizer import run_harmonization_raw;
run_harmonization_raw()"
```

12.5 Verifying the Installation

The project ships a 23-test automated suite covering the three components most prone to silent regressions in this specific codebase: the harmonizer's alias-matching logic, including two regression tests run against the real data/harmonization_entities.csv rather than a synthetic copy, guarding against a case-sensitivity bug that was once silently reintroduced by a data-file update rather than a code change; the Gemini client's response parsing and retry/backoff logic, with the network call itself mocked out so the suite never depends on a live API key; and the similarity-search wiring, tested against a small fixed dataset and a fake client. Run it with:

```
pip install -e ".[dev]"
pytest -q
pytest -q --cov=src --cov=config --cov-report=term-missing
ruff check .
```

Expected output is 23 passed in ~2s and All checks passed!; this exact pass/fail gate is also what the CI workflow (.github/workflows/ci.yml) checks on every push, and it will not build or publish the Docker image (§12.6) unless both pass.

For an end-to-end check that does not depend on the test suite, add one new row to data/Dataset_Raw.xlsx: any unused Conversation_ID, Type set to user, and a short question worded however you like; then, with the app running, search for the same question reworded completely differently, swapping every abbreviation for its full name if the original used any. A correct installation returns the row just added as a high-scoring match, typically 85% or higher, with a "why it matches" reasoning chip naming the specific terms it recognized as equivalent; this is the fastest way to confirm harmonization and the Gemini matching step are both genuinely working, not just that the server started.

12.6 Deploying to Docker / Portainer (University Data Lab)

The app calls out to OpenRouter for all model inference and performs no local GPU work, so per the university's "App deployment for Case Study projects" guide it only needs the Frontend / Non-GPU path: a single Docker container running this Dash app, deployed through Portainer. No backend notebook, GPU, or tunnel (LocalTunnel / Cloudflare Tunnel) is required.

1. CI builds and pushes the image automatically. Every push to main that passes lint and tests (§12.5) triggers .github/workflows/ci.yml, which builds the image from the Dockerfile and pushes it to GitHub Container Registry at ghcr.io/srh-heidelberg-university-adsa/similaritysearch:latest (and :<commit-sha>; check the repository's Packages tab on GitHub to confirm the exact reference after the first run; GHCR requires the name to be all lowercase). Because the repository is private, the pushed image is private too, so Portainer's default "Docker Hub (anonymous)" registry option cannot pull it: add ghcr.io as a custom registry first, under Portainer → Registries → Add registry → Custom registry, with URL ghcr.io, your GitHub username, and a GitHub personal access token with the read:packages scope as the password.

2. Create the container. In Portainer's local environment, under Containers → + Add container, set the fields below:

Field	Value
Name	e.g. similar-question-search
Registry	the ghcr.io custom registry added in step 1
Image	ghcr.io/srh-heidelberg-university-adsa/similaritysearch:latest
Publish all exposed ports to random host ports	On (maps the container's 8050 to a random public port)
Access control	Restricted, so teammates can also manage and restart it

3. Set the API key before deploying. Under Advanced container settings → Env, add GEMINI_API_KEY=<the real OpenRouter key>, either directly or by loading it from a .env file. This is the only place in the whole deployment the real key should exist: .dockerignore already excludes .env from the image itself, and .gitignore excludes it from the repository, so it is never baked in or committed.

4. Persist logs and outputs. Under Advanced container settings → Volumes, mount the paths below to persistent folders on the Data Lab host. Without this step, the container filesystem, and the audit log it holds (§6, usage_log.py) is wiped on every restart or redeploy.

Container path	Host path (example)
/app/logs	a persistent folder on the Data Lab host, e.g. /data/similar-question-

Container path	Host path (example)
	search/logs
/app/outputs	e.g. /data/similar-question-search/outputs

The container runs as an unprivileged user with UID/GID 1000 (set in the Dockerfile); if the mounted host folders show permission errors, chown them to UID 1000 on the host, or ask the Data Lab admin how volume ownership is handled on that server.

5. Deploy and confirm. Click Deploy the container, then in the container list scroll right to Published Ports to find the mapped host port and click it to confirm the app loads in a browser. Pick a subdomain (for example similar-question-search.app.data-lab.site) and send it, together with the port number, to Prof. Vu for confirmation, per the university's deployment guide.

12.7 Troubleshooting Common Setup Issues

- **Blank page, no error, after `python app.py`.** This is almost always the Dash / dash-mantine-components version mismatch documented in pyproject.toml: dash-mantine-components 2.x's own published metadata only claims dash>=2, but was verified by hand to render a blank page with Dash 2.x. Confirm pip show dash reports 4.x or later.
- **"Invalid API key" or 401 errors despite a key being set.** Double-check the key is an OpenRouter key (openrouter.ai/keys), not a Google AI Studio key; see §12.1's note. The two look superficially similar but are not interchangeable.
- **Portainer can't pull the image.** The repository, and therefore the image, is private. Confirm the ghcr.io custom registry (not the default Docker Hub option) is selected, and that the personal access token used as its password has the read:packages scope.
- **Permission denied writing to `logs/` or `outputs/` in a container deployment.** The container runs as UID/GID 1000; chown the mounted host folders to match, or ask the Data Lab admin how volume ownership is handled there.
- **Audit log or harmonization outputs already present after a fresh clone.** logs/ and outputs/*.csv / *.txt are intentionally committed in this submission as evidence the pipeline runs end to end, for academic review (§9). See the comment in .gitignore for how to re-exclude them before a genuine bank deployment, where a fresh install should start with empty logs.

Appendix A: Project Structure Reference

Path	Role
app.py	Dash + Mantine UI, the only entry point an evaluator interacts with directly (§7).
requirements.txt	Pinned minimum dependency versions: pandas, openpyxl, python-dotenv, openai, dash, dash-mantine-components.
.env / .env.example	Local, untracked environment file holding GEMINI_API_KEY (§6.1, §8.2).
config/settings.py	Single source of truth for every path, column

Path	Role
	name, and tunable constant (§8.1).
data/Dataset_Raw.xlsx	The raw, unharmonized conversation log, the one real input dataset.
data/harmonization_entities.csv	Canonicalization dictionary for divisions, financial terms, regions, regulators, companies, and related entities (§4.1).
data/harmonization_time_formats.csv	Canonicalization dictionary for months, quarters, half-years, full years, relative time expressions, and number-format conventions (§4.1).
instructions/similarity_instructions.txt	The full plain-text matching and scoring rubric sent to the model as its system message (§5.2).
outputs/Dataset_Harmonized.csv	The harmonized dataset (637 rows) produced by the batch pipeline, what the live search actually reads (§3.1, §4.5).
outputs/dataset_harmonization_qa_report.txt	Human-readable summary of every substitution made by the most recent harmonization run, sorted by frequency (§4.5).
src/harmonizer/harmonizer.py	Pure-function harmonization library: dictionary loading, alias-map construction, regex compilation, text substitution, QA reporting (§4.2–§4.3, §4.5).
src/harmonizer/__init__.py	Application layer over the harmonizer: file-path wiring, the four-step batch pipeline, and deterministic Question_Number assignment (§4.4).
src/llm/gemini_client.py	The sole module that talks to the model API: OpenRouter routing, structured output, rate limiting, retry/backoff (§6).
src/search/similarity_search.py	Live-search orchestrator: prompt construction, no-chunking single-call design, raw/harmonized text separation, result cleanup (§5).